Lecture 6: Graphs and Digraphs
Data Structures and Depth-First Search

Chapter 3 Notes

1 Introduction to Graphs

1.1 Basic Definitions

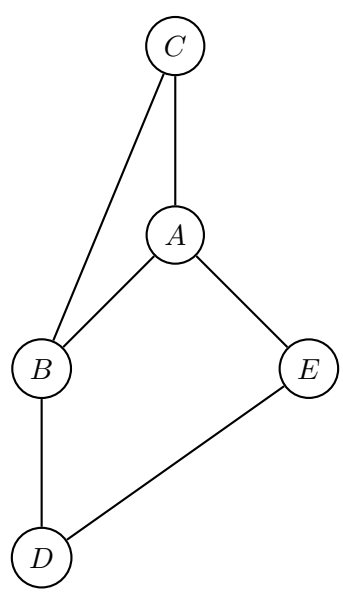
Definition 1 (Graph)**.** *A graph $G = (V, E)$ consists of:*

- *A set V of **vertices** (or nodes)*
- *A set E of **edges**, which are unordered pairs of distinct elements of V*

We have $E \subseteq \binom{V}{2}$, where $\binom{V}{2}$ denotes all 2-element subsets of V.

Example 1. *Consider a graph with $V = \{A, B, C, D, E\}$ and edges:*

$$E = \{(A, B), (A, C), (A, E), (B, C), (B, D), (D, E)\}$$



1.2 Why Use Graphs?

Graphs capture **pairwise relationships** between objects. Examples include:

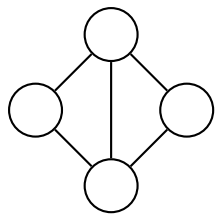
- **Road Network**: Vertices represent cities (e.g., Atlanta, Chattanooga, Nashville), edges represent roads connecting them

- **Map of Countries**: Vertices are countries, edges connect countries that share a border (e.g., USA-Canada, USA-Mexico)
- **Computer Network**: Vertices are computers/routers, edges represent network connections
- **Social Network**: Vertices are people, edges connect two people if they are friends

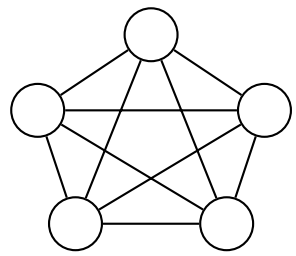
1.3 Planar Graphs

Definition 2 (Planar Graph)**.** *A graph is **planar** if it can be drawn on a page so that no edges cross.*

Planar:



Not Planar (K_5):



Theorem 1 (Four Color Theorem)**.** *Every planar graph can be properly colored with at most 4 colors, where vertices that share an edge get different colors.*

You can color a map with 4 colors so no two countries sharing a border get the same color.

1.4 Augmenting Graphs

We can augment a graph with additional information:

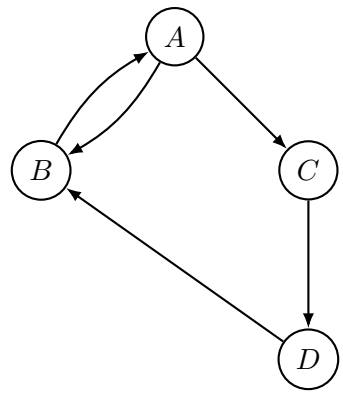
1. Give **weights** to edges (e.g., distances, costs)
2. Give **weights** to vertices (e.g., populations, capacities)
3. Allow for **multiple edges** between vertices or **self-loops**
4. Give edges a **direction**

1.5 Directed Graphs

Definition 3 (Directed Graph (Digraph))**.** *A directed graph (or digraph) $G = (V, E)$ consists of:*

- *A set V of vertices*
- *A set E of edges, which are **ordered pairs** of distinct elements of V*

Example 2. *Consider $V = \{A, B, C, D\}$ and $E = \{(A, B), (B, A), (A, C), (D, B), (C, D)\}$*



What are some situations that are best modeled by a directed graph instead of an undirected graph?

2 Data Structures for Graphs

There are several ways to store and access graphs computationally.

By convention we usually set $n = |V|$ and $m = |E|$ and report running times as functions of both n, m.

We will also assume in this class that vertex labels take $O(1)$ space to store.

2.1 Adjacency Matrix

Definition 4 (Adjacency Matrix)**.** *For a graph $G = (V, E)$ with $|V| = n$ vertices, the* ***adjacency matrix*** *$A(G)$ is an $n \times n$ matrix where:*

$$A_{ij} = \begin{cases} 1 & \text{if } (i,j) \in E \\ 0 & \text{if } (i,j) \notin E \end{cases}$$

For an undirected graph, the adjacency matrix is **symmetric** (i.e., $A_{ij} = A_{ji}$).

Example 3. *For the graph with vertices $V = \{A, B, C, D, E\}$ shown earlier:*

$$A(G) = \begin{array}{c|ccccc} & A & B & C & D & E \\ \hline A & 0 & 1 & 1 & 0 & 1 \\ B & 1 & 0 & 1 & 1 & 0 \\ C & 1 & 1 & 0 & 0 & 0 \\ D & 0 & 1 & 0 & 0 & 1 \\ E & 1 & 0 & 0 & 1 & 0 \end{array}$$

Properties:

- **Space complexity**: $O(n^2)$ where $n = |V|$
- **Time to query** whether a specific edge is present: $O(1)$
- Assumes $O(1)$ space to store a vertex label

Adjacency Matrix for Digraphs: For directed graphs, the matrix is generally **not symmetric**.

2.2 Adjacency Lists

Definition 5 (Adjacency List)**.** *For each vertex $v \in V$, store a list of all edges incident to v.*

Example 4. *For our example graph:*

$$\begin{aligned} L_A &= \{(A,B),(A,C),(A,E)\} \\ L_B &= \{(A,B),(B,C),(B,D)\} \\ L_C &= \{(A,C),(B,C)\} \\ L_D &= \{(B,D),(D,E)\} \\ L_E &= \{(A,E),(D,E)\} \end{aligned}$$

Note: Every edge appears in exactly 2 lists (one for each endpoint).
Properties:

- **Query time** for edge (u,v): $O(d_v) = O(n)$ where d_v is the degree of v
- **Space complexity**: $O(n+m)$ where $n = |V|$ and $m = |E|$
- If $m = o(n^2)$ (sparse graph), this is better than adjacency matrix

Adjacency Lists for Digraphs:
For directed graphs, we can use two lists for each vertex:

- L_v^+: forward edges (edges leaving v)
- L_v^-: backward edges (edges entering v)

Example 5. *For the digraph example:*

$$\begin{aligned} L_A^+ &= \{(A,B),(A,C)\} & L_A^- &= \{(B,A)\} \\ L_B^+ &= \{(B,A)\} & L_B^- &= \{(A,B),(D,B)\} \\ L_C^+ &= \{(C,D)\} & L_C^- &= \{(A,C)\} \\ L_D^+ &= \{(D,B)\} & L_D^- &= \{(C,D)\} \end{aligned}$$

Space complexity: $O(n+m)$

3 Exploring Graphs

3.1 Paths and Connectivity

Definition 6 (Path)**.** *A **path** in a graph G is a sequence of vertices $v_1, v_2, \ldots, v_k$ such that for $i = 1, \ldots, k-1$, we have $(v_i, v_{i+1}) \in E$.*

Definition 7 (Simple Path)**.** *A **simple path** is one where all vertices in the sequence are distinct.*

Definition 8 (Connected Vertices)**.** *Vertices $u, v \in V$ are **connected** if there is a path such that the first vertex in the sequence is u and the last is v.*

Definition 9 (Connected Graph)**.** *A graph G is **connected** if u and v are connected for all $u, v \in V$.*

Definition 10 (Subgraph)**.** *A **subgraph** H of a graph $G = (V,E)$ is a graph $H = (V_H, E_H)$ such that $V_H \subseteq V$ and $E_H \subseteq E$.*

Definition 11 (Connected Component)**.** *A **connected component** of G is a maximal connected subgraph of G.*

3.2 Algorithmic Problems

Given a graph G:

1. Determine if vertices u and v are connected
2. Determine if G is connected
3. Count or enumerate the connected components of G

4 Depth-First Search (DFS)

4.1 The Explore Subroutine

The basic building block of DFS is the `Explore` procedure:

Algorithm 1 Explore(G, v)

Require: Graph $G = (V, E)$, vertex $v \in V$
Ensure: Mark `visited`(u) = true for all u connected to v

```
visited(v) ← true
label(v) ← cc                                   ▷ Label with current component number
previsit(v)                                     ▷ Optional action
for each (v, u) ∈ E do
    if not visited(u) then
        Explore(G, u)
    end if
end for
postvisit(v)                                    ▷ Optional action
```

4.2 Full DFS Algorithm

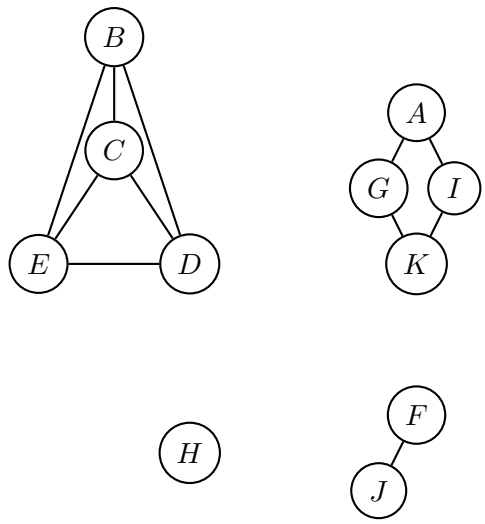
Algorithm 2 DFS(G)

Require: Graph $G = (V, E)$
Ensure: All vertices labeled by connected component

```
for all v ∈ V do
    visited(v) ← false
end for
cc ← 0                                          ▷ Connected component counter
for v ∈ V do
    if not visited(v) then
        cc ← cc + 1
        Explore(G, v)
    end if
end for
return cc                                       ▷ Number of connected components
```

4.3 Example Execution

Consider the following graph:



Execution trace (assuming vertices processed in order $A, B, C, \ldots$):

- `visited`(A) = false, `cc` $= 1$, `Explore`(G, A)
- Explore marks A, G, I, K as visited with label 1
- `visited`(B) = false, `cc` $= 2$, `Explore`(G, B)
- Explore marks B, C, D, E as visited with label 2
- `visited`(C, D, E) = true (skip)
- `visited`(F) = false, `cc` $= 3$, `Explore`(G, F)
- Explore marks F, J as visited with label 3
- `visited`$(G, H, I, J, K) = \ldots$
- `visited`(H) = false, `cc` $= 4$, `Explore`(G, H)
- Explore marks H as visited with label 4
- Output: `cc` $= 4$ connected components

4.4 Time Complexity

When using the adjacency list representation:

Runtime: $O(n + m)$

- Each vertex is visited exactly once
- Each edge is examined at most twice (once from each endpoint)
- Total time is linear in the size of the graph

This is optimal since we need to at least look at all vertices and edges to determine connectivity.

5 Summary

- Graphs model pairwise relationships and are fundamental in computer science
- Two main data structures: adjacency matrix ($O(n^2)$ space) and adjacency lists ($O(n + m)$ space)
- Adjacency lists are preferred for sparse graphs
- DFS can find and label all connected components in $O(n + m)$ time